Cours 06 bis — Vérité, conditions et boucle de jeu : « Devine mon nombre »

> **Avant** : cours 06 (le `if`, les conditions). **Comment travailler** : ce cours supplémentaire n'utilise pas openFrameworks. Ce sont des **programmes console** : du texte qui s'affiche, des réponses tapées au clavier. Dans Visual Studio, crée un projet *Application console* (Fichier → Nouveau → Projet → **Application console** C++) et écris chaque programme dans son `main.cpp`, un programme à la fois. Lance avec `F5`, comme d'habitude.

Au cours 06, tu as utilisé le `if` pour faire rebondir une forme. Ce cours creuse tout ce qui tourne autour : ce que *vrai* et *faux* veulent dire pour la machine, le `else`, le `switch`, les boucles `while` et `do`, les opérateurs logiques `&&`, `||` et `!`, et le hasard version C++ standard. À la fin, tu assembles tout dans un vrai petit jeu : **« Devine mon nombre »** — l'ordinateur choisit un nombre, tu le devines, il te guide.

Pourquoi la console ? Parce que ces mécanismes sont du **C++ pur** : ils marchent partout, openFrameworks compris. Ici, pas de dessin — juste toi, le programme, et la logique. Chaque programme du cours est suivi d'un **Exercice** à écrire toi-même : une suite du programme que tu viens de taper, ou un nouveau petit programme guidé par étapes. C'est là que ça rentre — ne les saute pas.

1. Le vrai et le faux

Tout ce qui suit repose sur une seule idée : une **expression** peut être évaluée comme **vraie** ou **fausse**. C++ a un type pour ça, le `bool` (booléen), et deux mots-clés :

```
bool enVie { true };
bool partieFinie { false };
```

Une variable `bool` tient dans une toute petite boîte et ne stocke que ça : vrai ou faux. On peut aussi y ranger le résultat d'une comparaison :

```
bool aGagne = (score >= 1000);
```

Les opérateurs de comparaison

Pour fabriquer du vrai ou du faux, on **compare** des valeurs. Six opérateurs :

Opérateur	Se lit	Exemple vrai	Exemple faux
`==`	est égal à	`5 == 5`	`5 == 8`
`!=`	est différent de	`5 != 8`	`5 != 5`
`>`	est plus grand que	`8 > 5`	`5 > 8`
`<`	est plus petit que	`5 < 8`	`8 < 5`

Opérateur	Se lit	Exemple vrai	Exemple faux
`>=`	plus grand ou égal	`8 >= 5`	`5 >= 8`
`<=`	plus petit ou égal	`5 <= 8`	`8 <= 5`

Toute valeur a une vérité

C++ va plus loin : **n'importe quelle valeur** peut être interprétée comme vraie ou fausse. La règle est simple :

- toute valeur **différente de zéro** est interprétée comme `true` ;
- **zéro** est interprété comme `false`.

Donc `score` tout seul, dans un endroit qui attend du vrai/faux, veut dire « `score` n'est pas zéro ». Tu vas le voir en action tout de suite.

> **Essaie** : sur papier, donne la vérité (vrai ou faux) de : `3 > 7`, `10 == 10`, `10 = 10` (piège !), `0`, `-42`, `7 != 7`. Vérifie plus bas dans le cours.

2. Étape 1 — le `if` : la jauge de score

Le `if` exécute une instruction **seulement si** une expression est vraie :

```
if (expression)
    instruction;
```

Si `expression` est vraie, `instruction` est exécutée ; sinon elle est sautée, et le programme continue après. Voici un programme complet qui commente un score de jeu — tape-le et lance-le :

```cpp
// Jauge de score
// Le if sous toutes ses formes

#include <iostream>
using namespace std;

int main()
{
    if (true)
    {
        cout << "Ceci s'affiche toujours.\n\n";
    }

    if (false)
    {
        cout << "Ceci ne s'affiche jamais.\n\n";
    }

    int score { 1000 };
    if (score)
    {
        cout << "Au moins, ton score n'est pas zero.\n\n";
    }

    if (score >= 250)
    {
        cout << "Tu as fait au moins 250. Correct.\n\n";
    }

    if (score >= 500)
    {
        cout << "Tu as fait au moins 500. Joli.\n\n";
        if (score >= 1000)
        {
            cout << "Tu as fait au moins 1000. Impressionnant !\n";
        }
    }

    return 0;
}
```

Tout s'affiche, sauf la deuxième ligne. Relis chaque `if` :

- `if (true)` : l'expression est littéralement vraie, l'instruction est toujours exécutée. (Aucun intérêt en vrai, c'est pour la démonstration.)
- `if (false)` : jamais exécutée.
- `if (score)` : `score` vaut 1000, différent de zéro, donc interprété comme vrai. C'est la règle de la section 1.
- `if (score >= 250)` : une vraie comparaison, le cas d'usage normal.
- Les deux derniers `if` sont **imbriqués** : le test `score >= 1000` n'est même pas regardé si `score >= 500` est faux. Imbriquer est permis à volonté — mais au-delà de deux ou trois niveaux, le code devient dur à suivre : c'est le signe qu'il faut réorganiser.

Piège : pas de point-virgule après la parenthèse du `if` ! `if (score >= 250);` compile… mais le `;` est une instruction **vide** : c'est elle qui est conditionnée, et la ligne suivante s'exécute toujours. Erreur classique et silencieuse.

Astuce : l'instruction d'un `if` peut être un **bloc** — plusieurs instructions entre accolades `{ }`, comme ci-dessus. Prends l'habitude de toujours mettre les accolades, même pour une seule ligne : tu éviteras des surprises le jour où tu en ajoutes une deuxième.

Piège : `=` affecte, `==` compare — tu l'as vu au cours 06, le voici en console. `if (score = 1000)` **range** 1000 dans `score`, et la valeur de l'affectation est 1000… donc vrai : le `if` passe toujours, quel que soit le score de départ. Le compilateur ne dit (presque) rien. Relis tes `if` !

Essaie : change `int score { 1000 };` en `{ 0 }` puis en `{ 300 }` et prédis, *avant de lancer*, quelles lignes s'afficheront.

Exercice — la jauge enrichie. En suite du programme : ajoute un palier à 2000, « Tu as fait au moins 2000. Legendaire ! », au bon endroit dans l'imbrication — il ne doit être testé que si les paliers 500 et 1000 sont déjà passés. Puis ajoute un `if` qui affiche « Zero pointe... » quand le score est **exactement** zéro. Attention : `if (score)` dit le contraire de ce que tu veux — quelle comparaison te faut-il ?

3. Étape 2 — le `else` : sinon

Un `if` sait faire quelque chose quand c'est vrai. Le `else` ajoute le **sinon** :

```
if (expression)
    instruction1;
else
    instruction2;
```

Si l'expression est vraie, `instruction1` ; sinon, `instruction2`. Exactement une des deux s'exécute. La jauge de score, version 2.0 — cette fois c'est toi qui entres le score :

```
// Jauge de score 2.0
// Le else, et une saisie clavier

#include <iostream>
using namespace std;

int main()
{
    int score;
    cout << "Entre ton score : ";
    cin >> score;

    if (score >= 1000)
    {
        cout << "Impressionnant !\n";
    }
    else
    {
        cout << "Moins impressionnant.\n";
    }

    return 0;
}
```

Nouveauté au passage : `cin >> score` **lit** ce que tu tapes au clavier et le range dans `score`. C'est le symétrique de `cout <<` qui écrit ; les chevrons pointent dans le sens où la donnée circule.

> **Essaie** : lance le programme plusieurs fois avec des scores des deux côtés de 1000. Puis teste 1000 pile — de quel côté tombe-t-il, et pourquoi ?

> **Exercice — pair ou impair** *(nouveau programme)* :
>
> 1. lis un entier `n` au clavier, comme dans la jauge 2.0 ;
> 2. outil fourni : `n % 2` donne le **reste** de la division de `n` par 2 — 0 si `n` est pair, autre chose sinon ;
> 3. avec un `if` / `else`, affiche « pair » ou « impair ».
>
> Teste avec des grands nombres, zéro, et des négatifs.

4. Étape 3 — enchaîner les cas : `else if`

Pour tester **une série** d'expressions et ne réagir qu'à la première qui est vraie, on enchaîne :

```
if (expression1)
    instruction1;
else if (expression2)
    instruction2;
else if (expression3)
    instruction3;
else
    instruction4;
```

Dès qu'une expression est vraie, son instruction s'exécute et **tout le reste est sauté**. La jauge de score 3.0, avec une réponse par palier :

```
// Jauge de score 3.0
// La suite de else if

#include <iostream>
using namespace std;

int main()
{
    int score;
    cout << "Entre ton score : ";
    cin >> score;

    if (score >= 1000)
    {
        cout << "Impressionnant !\n";
    }
    else if (score >= 500)
    {
        cout << "Joli.\n";
    }
    else if (score >= 250)
    {
        cout << "Correct.\n";
    }
    else
    {
        cout << "Ca ira mieux la prochaine fois.\n";
    }

    return 0;
}
```

Compare avec la version 1 : là-bas, un score de 1000 déclenchait *tous* les messages ; ici, un seul — le premier palier atteint. L'**ordre** des tests compte : du plus exigeant au moins exigeant.

Astuce : le `else` final attrape **tout ce qui reste**. C'est le filet de sécurité de la chaîne : mets-y le cas par défaut.

Essaie : inverse l'ordre des paliers (250 d'abord, puis 500, puis 1000). Que répond le programme à un score de 2000 ? Explique.

> **Exercice — la mention** *(nouveau programme)*, sur le modèle de la jauge 3.0 : lis une note sur 20 et affiche la mention — « Tres bien » à partir de 16, « Bien » à partir de 14, « Assez bien » à partir de 12, « Passable » à partir de 10, « Recale » sinon. Deux questions à régler *avant* d'écrire : dans quel ordre ranger les paliers, et que répond ton programme à une note de 25 ? (Bonus : un `if` de garde avant la chaîne peut signaler les notes au-dessus de 20.)

5. Étape 4 — le `switch` : le menu de difficulté

Quand on compare **une même valeur** à plusieurs possibilités précises, C++ offre une écriture dédiée :

```
switch (expression)
{
case valeur1:
    instructions1;
    break;
case valeur2:
    instructions2;
    break;
default:
    instructionsParDefaut;
}
```

Le `switch` regarde la valeur de l'expression et saute au `case` correspondant ; `default` attrape tout le reste ; `break` sort du `switch`. Un menu de jeu classique :

```
// Menu de difficulte
// Le switch

#include <iostream>
using namespace std;

int main()
{
    cout << "Niveaux de difficulte\n\n";
    cout << "1 - Facile\n";
    cout << "2 - Normal\n";
    cout << "3 - Difficile\n\n";

    int choix;
    cout << "Ton choix : ";
    cin >> choix;

    switch (choix)
    {
    case 1:
        cout << "Tu as choisi Facile.\n";
        break;
    case 2:
        cout << "Tu as choisi Normal.\n";
        break;
    case 3:
        cout << "Tu as choisi Difficile.\n";
        break;
    default:
        cout << "Ce choix n'existe pas.\n";
    }

    return 0;
}
```

Tu reconnais la structure : c'est le clavier du cours 11 qui t'attend (`switch` sur la touche pressée pour choisir un filtre).

> **Piège** : le `switch` ne teste que des valeurs qui se ramènent à un **entier** (`int` , `char` …). Pas de `float` , pas de texte, pas de `score >= 500` : pour des plages ou des comparaisons, c'est la suite de `else if` qu'il faut.

> **Piège** : oublier un `break`, ça compile — et l'exécution **continue dans le case suivant**. Choisis 1 après avoir enlevé les `break` : le programme affiche les quatre messages. Parfois on le fait exprès (regrouper deux cas) ; le plus souvent c'est un bug.

> **Astuce** : mets toujours un `default`, même si tu crois avoir couvert tous les cas — c'est lui qui attrape la faute de frappe de l'utilisateur.

> **Essaie** : ajoute un niveau `4 - Cauchemar`, puis enlève le `break` du case 3 pour voir la « chute » dans le case 4.

> **Exercice — la calculatrice** *(nouveau programme)* :
>
> 1. affiche un menu : 1 - addition, 2 - soustraction, 3 - multiplication, 4 - division ;
> 2. lis le choix, puis deux nombres `float a` et `float b` ;
> 3. un `switch` sur le choix calcule et affiche le résultat de l'opération ; le `default` refuse les choix inconnus ;
> 4. cas particulier : pour la division, un `if` doit refuser `b` égal à zéro **avant** de diviser.

6. Étape 5 — la boucle `while` : encore une partie ?

Le `for` du cours 02 répète un nombre de tours **connu d'avance**. Le `while` répète **tant qu'une expression est vraie** — sans savoir combien de fois :

```
while (expression)
    instruction;
```

Avant chaque tour, l'expression est évaluée : vraie, on refait un tour ; fausse, on passe à la suite. Le squelette de tout jeu qui propose de rejouer :

```
// Encore une partie ?
// La boucle while

#include <iostream>
using namespace std;

int main()
{
    char encore { 'y' };
    while (encore == 'y')
    {
        cout << "**Une partie de jeu palpitante**\n";
        cout << "Tu veux rejouer ? (y/n) : ";
        cin >> encore;
    }
    cout << "\nD'accord, a bientot !\n";

    return 0;
}
```

Le type `char` stocke **un caractère** (une lettre, un chiffre, un signe), entre apostrophes : `'y'`. Remarque le détail qui compte : `encore` est initialisée à `'y'` **avant** la boucle. Sans ça, le test du premier tour lirait une variable au contenu inconnu — la boucle pourrait ne jamais démarrer.

> **Essaie** : réponds `n` du premier coup, puis `y` trois fois. Puis initialise `encore` à `'n'` : que se passe-t-il ?

> **Exercice — la caisse enregistreuse** *(nouveau programme)* : additionner des prix jusqu'à ce qu'on tape 0.
>
> 1. deux variables : `float total { 0 };` et un `float prix` ;
> 2. lis un premier prix **avant** la boucle — comme `encore` était initialisée avant la sienne ;
> 3. `while (prix != 0)` : ajoute le prix au total, puis lis le prix suivant ;
> 4. après la boucle, affiche le total.
>
> La partie subtile est le point 2 : sans cette première lecture, que testerait le premier tour ?

7. Étape 6 — la boucle `do` : jouer d'abord, demander ensuite

Variante : le `do` évalue l'expression **après** le tour, si bien que le corps s'exécute **au moins une fois** :

```
do
    instruction;
while (expression);
```

La version 2.0 du programme précédent — plus naturelle, puisqu'un jeu devrait se jouer au moins une fois avant de demander si on rejoue :

```
// Encore une partie ? 2.0
// La boucle do

#include <iostream>
using namespace std;

int main()
{
    char encore;
    do
    {
        cout << "**Une partie de jeu palpitante**\n";
        cout << "Tu veux rejouer ? (y/n) : ";
        cin >> encore;
    } while (encore == 'y');

    cout << "\nD'accord, a bientot !\n";

    return 0;
}
```

Plus besoin d'initialiser `encore` : elle est remplie par le `cin` avant d'être testée.

Dans la vraie vie, les programmeurs utilisent bien plus souvent `while` que `do` : la boucle qui annonce son test **en tête** se lit mieux. Mais quand « au moins une fois » est exactement ce qu'on veut, le `do` est l'outil juste.

Piège : la **boucle infinie**. Si l'expression ne devient jamais fausse, le programme tourne pour toujours :

```
int test { 10 };
while (test == 10)
{
    cout << "Encore et encore...\n";
}
```

Rien dans le corps ne modifie `test` : le test est vrai pour l'éternité. Le compilateur ne détecte **pas** ça — c'est une erreur de logique, pas de syntaxe. En console, `Ctrl+C` (ou le bouton Stop de Visual Studio) tue le programme. Règle d'or : dans une boucle `while`, quelque chose doit pouvoir **changer** l'expression testée.

Essaie : provoque volontairement la boucle infinie ci-dessus, constate, arrête le programme. Mieux vaut la rencontrer maintenant, exprès, que plus tard, par accident.

Exercice — le digicode *(nouveau programme)* : la porte s'ouvre au code 1234.

1. dans un `do`, demande le code et lis-le ;
2. compte chaque tentative au passage ;
3. `while (code != 1234)` : on redemande tant que c'est faux ;
4. après la boucle, affiche « Ouvert en N tentatives ».

Et explique : pourquoi `do` est-il ici un meilleur choix que `while` ?

8. Étape 7 — `break` et `continue` : le compteur capricieux

Deux instructions modifient le déroulement d'une boucle de l'intérieur :

- `break` : **sortir** de la boucle immédiatement ;
- `continue` : sauter directement au **tour suivant**.

Un compteur qui compte de 1 à 10, mais boude le 5 :

```
// Compteur capricieux
// break et continue

#include <iostream>
using namespace std;

int main()
{
    int compteur { 0 };
    while (true)
    {
        compteur += 1;

        // sortir de la boucle au-dela de 10
        if (compteur > 10)
        {
            break;
        }

        // sauter le 5
        if (compteur == 5)
        {
            continue;
        }

        cout << compteur << endl;
    }

    return 0;
}
```

Affichage : 1 2 3 4 6 7 8 9 10. `while (true)` est une boucle **volontairement** infinie — c'est le `break` qui en sort. Ce motif est courant pour la boucle principale d'un jeu ; on y revient à la section 11. Et `compteur += 1;` est le raccourci de `compteur = compteur + 1;`.

Astuce : `while (true) + break`, c'est puissant mais à consommer avec modération : la condition de sortie n'est plus visible en tête de boucle, il faut la chercher dans le corps. Quand une condition simple suffit (`while (compteur <= 10)`), préfère-la. De même, `break` et `continue` disséminés partout rendent une boucle imprévisible — souvent, on peut les remplacer en repensant le test.

Essaie : réécris le compteur sans `while (true)` ni `break` (le test de fin passe dans le `while`), en gardant le `continue` pour le 5.

> **Exercice — l'addition qui déborde.** En suite du compteur : un programme qui additionne 1 + 2 + 3 + … et s'arrête dès que la somme **dépasse 100**. Même squelette que le compteur capricieux : un `while (true)`, un compteur qui monte, une somme qui accumule, un `break` quand la somme dépasse. À la sortie, affiche la somme atteinte et le dernier nombre ajouté. Question bonus : réécris-le sans `while (true)` — quelle condition mets-tu en tête de boucle ?

9. Étape 8 — les opérateurs logiques : le réseau des designers

Une condition simple ne suffit pas toujours : « le bon utilisateur **et** le bon mot de passe », « guest en identifiant **ou** en mot de passe ». Trois opérateurs combinent des expressions vraies/fausses :

Opérateur	Se lit	Vrai quand…
`!`	NON	l'expression est fausse
`&&`	ET	les **deux** expressions sont vraies
`||`	OU	**au moins une** des deux est vraie

Le programme qui les utilise tous les trois — un écran de connexion pour un club très fermé de game designers :

```
// Le reseau des designers
// Les operateurs logiques

#include <iostream>
#include <string>
using namespace std;

int main()
{
    string utilisateur;
    string motDePasse;
    bool succes;

    cout << "\tBienvenue sur le Reseau des Designers\n\n";
    do
    {
        cout << "Identifiant : ";
        cin >> utilisateur;
        cout << "Mot de passe : ";
        cin >> motDePasse;

        if (utilisateur == "S.Meier" && motDePasse == "civilization")
        {
            cout << "\nSalut, Sid.\n";
            succes = true;
        }
        else if (utilisateur == "S.Miyamoto" && motDePasse == "mariobros")
        {
            cout << "\nQuoi de neuf, Shigeru ?\n";
            succes = true;
        }
        else if (utilisateur == "W.Wright" && motDePasse == "thesims")
        {
            cout << "\nComment ca va, Will ?\n";
            succes = true;
        }
        else if (utilisateur == "guest" || motDePasse == "guest")
        {
            cout << "\nBienvenue, invite.\n";
            succes = true;
        }
        else
        {
```

```
            cout << "\nConnexion refusee.\n";
            succes = false;
        }
    } while (!succes);

    return 0;
}
```

(Le type `string`, vu au cours 04, stocke du texte ; il vient de `#include <string>`.) Décortiquons les trois opérateurs sur ce programme.

Le ET : `&&`

`utilisateur == "S.Meier" && motDePasse == "civilization"` n'est vrai que si **les deux** comparaisons sont vraies : le bon identifiant *et* le bon mot de passe. Toutes les combinaisons :

`utilisateur == "S.Meier"`	`motDePasse == "civilization"`	**ET des deux**
vrai	vrai	**vrai**
vrai	faux	faux
faux	vrai	faux
faux	faux	faux

Le OU : `||`

`utilisateur == "guest" || motDePasse == "guest"` est vrai dès qu'**au moins une** des deux comparaisons est vraie : taper `guest` dans l'un ou l'autre champ suffit pour entrer en invité (les deux, ça marche aussi) :

`utilisateur == "guest"`	`motDePasse == "guest"`	**OU des deux**
vrai	vrai	**vrai**
vrai	faux	**vrai**
faux	vrai	**vrai**
faux	faux	faux

Le NON : `!`

`!succes` **inverse** la vérité : vrai si `succes` est faux, faux si `succes` est vrai.

`succes`	`!succes`
vrai	faux
faux	vrai

C'est lui qui pilote la boucle : `do { … } while (!succes);` se lit « recommence **tant que** la connexion n'a **pas** réussi ». Tant qu'un login échoue, `succes` est faux, `!succes` est vrai, on redemande ; dès qu'un login passe, `succes` devient vrai, `!succes` devient faux, la boucle s'arrête.

L'ordre d'évaluation

Comme les opérateurs arithmétiques, les opérateurs logiques ont des **priorités** : `!` passe avant `&&`, qui passe avant `||`. Et comme en arithmétique, les **parenthèses** forcent l'ordre que tu veux. Le meilleur conseil : écris des expressions **simples et claires** plutôt que de jouer au plus malin avec la table des priorités.

> **Astuce** : les *parenthèses redondantes* — qui ne changent rien au résultat — rendent souvent l'expression plus lisible :
>
> ```
> (utilisateur == "S.Meier" && motDePasse == "civilization")
> ( (utilisateur == "S.Meier") && (motDePasse == "civilization") )
> ```
>
> Les deux lignes signifient la même chose ; la deuxième fait mieux ressortir les deux comparaisons reliées par le `&&`. À toi de juger, cas par cas, si elles aident ou encombrent.

> **Essaie** : connecte-toi en Sid Meier, puis avec un mot de passe faux, puis en `guest` (dans un champ, dans l'autre, dans les deux). Ajoute ensuite ton propre compte au club.

> **Exercice — le videur** *(nouveau programme)* : l'entrée d'une salle de concert.
>
> 1. demande l'âge (`int`), puis « es-tu sur la liste ? (y/n) » et « as-tu une invitation ? (y/n) » (deux `char`) ;
> 2. la règle : on entre si on a **18 ans ou plus ET** qu'on est sur la liste, **OU** si on a une invitation. Écris-la en une seule expression, avec des parenthèses qui rendent la règle lisible ;
> 3. un `if` / `else` affiche « Entre » ou « Refuse » ;
> 4. avant de lancer, dresse un petit tableau à la main : trois cas qui doivent passer, trois qui doivent être refusés — puis vérifie ton programme dessus.

10. Étape 9 — le hasard : lancer un dé

Le hasard rend un jeu vivant : un adversaire imprévisible, une porte qui s'ouvre au mauvais moment — ou un nombre secret à deviner. Tu connais `ofRandom` (cours 03) ; voici la version **C++ standard**, celle qui marche aussi hors d'openFrameworks. Un lancer de dé à six faces :

```
// Lancer de de
// Generer des nombres aleatoires

#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main()
{
    srand(static_cast<unsigned int>(time(0))); // plante la graine

    int nombreAleatoire = rand(); // un nombre au hasard

    int lancer = (nombreAleatoire % 6) + 1; // un nombre entre 1 et 6
    cout << "Tu as fait un " << lancer << endl;

    return 0;
}
```

Trois lignes à comprendre.

`rand()` : tirer un nombre

`#include <cstdlib>` apporte, entre autres, la fonction `rand()`, qui renvoie un nombre entier au hasard entre 0 et au moins 32767. On appelle la fonction, on range ce qu'elle renvoie — comme n'importe quelle fonction qui renvoie une valeur. `rand()` ne prend aucun argument, d'où les parenthèses vides.

> **Astuce** : la borne haute exacte de `rand()` dépend de ton compilateur ; elle est rangée dans la constante `RAND_MAX` (définie dans `cstdlib`). Pour la connaître : `cout << RAND_MAX;`.

`srand(...)` : planter la graine

Tu l'as appris au cours 03 : l'ordinateur ne tire pas de « vrais » nombres au hasard, il calcule une suite **pseudo-aléatoire** à partir d'un point de départ, la **graine**. Sans graine, la suite est la même à chaque lancement du programme — le dé ferait toujours les mêmes scores, comme un livre de nombres qu'on rouvrirait toujours à la page 1.

```
srand(static_cast<unsigned int>(time(0))); // plante la graine
```

`srand(...)` plante la graine, et on lui donne `time(0)` (de `#include <ctime>`) : un nombre construit à partir de la date et de l'heure — différent à chaque exécution, donc une suite différente à chaque partie. Le `static_cast<unsigned int>(...)` **convertit** ce nombre dans le type que `srand` attend ; retiens juste

l'idée (convertir un type en un autre), la ligne se recopie telle quelle. Une seule fois, au début du programme, avant les appels à `rand()`.

`% 6 + 1` : ramener dans une plage

`rand()` renvoie un nombre entre 0 et 32767 ; un dé va de 1 à 6. Le `%` (modulo, le reste de la division entière — cours 13 l'utilisera beaucoup) fait le pont :

- `nombreAleatoire % 6` : le reste de la division par 6, donc un nombre entre **0 et 5** ;
- `+ 1` : décale la plage à **1 à 6**.

La recette se généralise : `rand() % n + debut` donne un nombre entre `debut` et `debut + n - 1`.

> **Piège** : le modulo ne répartit pas toujours *parfaitement* les tirages — certaines valeurs de la plage peuvent sortir un poil plus souvent que d'autres. Pour nos jeux, c'est sans importance ; sache juste que ça existe.

> **Essaie** : lance le programme plusieurs fois (le dé change). Mets la ligne `srand(...)` en commentaire et relance plusieurs fois : le dé se répète — la démonstration exacte de la graine du cours 03. Puis fabrique un dé à 20 faces.

> **Exercice — pile ou face** *(nouveau programme)* : l'ordinateur lance une pièce, tu paries.
>
> 1. plante la graine — la ligne se recopie telle quelle ;
> 2. demande au joueur son pari : 0 pour pile, 1 pour face ;
> 3. tire la pièce avec `rand()` et la recette de la plage (quelle plage ?) ;
> 4. compare le pari et le tirage, annonce gagné ou perdu — et ce qu'avait donné la pièce.
>
> Version d'après : enferme tout ça dans une boucle « encore une partie ? » de l'étape 5.

11. La boucle de jeu

Prends n'importe quel jeu — un shoot spatial minimaliste, un RPG énorme : le **déroulement** est presque toujours le même. On l'appelle la **boucle de jeu** (*game loop*) :



- **Mise en place** : régler la partie, charger les ressources (sons, images), présenter l'histoire et l'objectif.
- **Lire l'entrée du joueur** : clavier, souris, manette…
- **Mettre à jour la logique** : appliquer les règles du jeu à ce qui vient de se passer — physique, adversaires, score.
- **Mettre à jour l'affichage** : montrer le nouvel état. Souvent la partie la plus coûteuse (le dessin !) — mais ça peut être une simple ligne de texte.
- **Partie finie ?** : si non, on reboucle sur l'entrée du joueur ; si oui, on sort.
- **Conclusion** : le score final, les adieux, le rangement.

Ce schéma doit te rappeler quelque chose : `setup()` / `update()` / `draw()` du cours 05, **c'est exactement ça** — openFrameworks fait tourner la boucle pour toi (lecture de la souris, appel d'`update`, appel de `draw`, et on recommence). En console, personne ne la fait tourner à ta place : tu vas l'écrire toi-même, avec un `do … while`.

12. Étape 10 — le projet : « Devine mon nombre »

Le jeu classique : l'ordinateur choisit un nombre entre 1 et 100, tu le devines en un minimum d'essais ; à chaque proposition, il répond « trop grand », « trop petit » ou « gagné ». Une partie ressemble à ça :

```
        Bienvenue dans Devine mon nombre

Propose un nombre : 50
Trop petit !

Propose un nombre : 75
Trop grand !

Propose un nombre : 63
C'est ca ! Trouve en 3 essais !
```

Relis ce jeu avec les lunettes de la boucle de jeu : mise en place = choisir le nombre secret ; entrée = lire la proposition ; logique = compter l'essai, comparer ; affichage = trop grand / trop petit ; partie finie ? = nombre trouvé ; conclusion = féliciter et donner le nombre d'essais. Tout y est. Construisons-le en trois morceaux.

La mise en place

Les includes, la graine, le nombre secret et deux variables de partie :

```
// Devine mon nombre
// Le classique jeu du nombre a deviner

#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main()
{
    srand(static_cast<unsigned int>(time(0))); // plante la graine

    int nombreSecret = rand() % 100 + 1; // entre 1 et 100
    int essais { 0 };
    int proposition;

    cout << "\tBienvenue dans Devine mon nombre\n\n";
```

Tu reconnais tout : la graine et la recette `% 100 + 1` de l'étape 9 ; `essais` démarre à zéro ; `proposition` sera remplie par le joueur. (`\t` insère une tabulation, `\n` un saut de ligne.)

La boucle de jeu

Un `do` — car il faut proposer au moins une fois — avec la suite `if` / `else if` / `else` de l'étape 3 :

```
    do
    {
        cout << "Propose un nombre : ";
        cin >> proposition;
        ++essais;

        if (proposition > nombreSecret)
        {
            cout << "Trop grand !\n\n";
        }
        else if (proposition < nombreSecret)
        {
            cout << "Trop petit !\n\n";
        }
        else
        {
            cout << "\nC'est ca ! Trouve en " << essais << " essais !\n";
        }
    } while (proposition != nombreSecret);
```

À chaque tour : lire la proposition, compter l'essai (`++essais` est encore un raccourci de `essais = essais + 1`), puis répondre — trop grand, trop petit, ou gagné. Le `while (proposition != nombreSecret)` reboucle tant que ce n'est pas trouvé : les `if` **imbriqués dans** la boucle, c'est le motif de tous les jeux.

La conclusion

Le nombre trouvé, la boucle est finie ; il ne reste qu'à quitter :

```
    return 0;
}
```

Assemble les trois morceaux dans `main.cpp`, lance, et joue. C'est un programme complet, écrit uniquement avec ce que ce cours a introduit : `rand` et la graine, `do … while`, `if` / `else if` / `else`, `!=`, un compteur.

> **Essaie** : joue plusieurs parties. Quelle est ta stratégie pour minimiser les essais ? (Indice : proposer 50, puis couper l'intervalle restant en deux à chaque coup — 7 essais suffisent toujours. C'est la *recherche dichotomique*, tu la recroiseras.)

> **Exercice — la partie limitée.** En suite du jeu : le joueur n'a droit qu'à **7 essais** (la dichotomie garantit que c'est jouable). Deux chantiers :

1. la boucle doit continuer tant que le nombre n'est pas trouvé **ET** qu'il reste des essais — reformule la condition du `while` avec un opérateur logique de l'étape 8 ;
2. après la boucle, il faut savoir *pourquoi* on est sorti : un `if` final compare `proposition` et `nombreSecret`, pour féliciter… ou révéler le nombre en se moquant gentiment.

Questions et réponses

Est-on obligé d'utiliser `true` et `false` ? Non, mais c'est la bonne pratique. Avant leur arrivée dans le langage, on utilisait 1 pour vrai et 0 pour faux ; maintenant qu'ils existent, autant écrire ce qu'on pense.

Peut-on mettre autre chose que `true` / `false` dans un `bool` ? Oui : n'importe quelle expression. Le `bool` stockera sa vérité — `bool aGagne = (score >= 1000);`.

Un `switch` peut-il tester du texte ou un `float` ? Non : seulement des valeurs interprétables comme des entiers (`char` inclus). Pour le reste, une suite de `if` / `else if`.

Une boucle infinie, c'est grave ? Le programme ne s'arrête plus tout seul ; il faut le tuer de l'extérieur. Et le compilateur ne peut pas la détecter : c'est une erreur de **logique**, à toi de la traquer.

Alors pourquoi `while (true)` est-il accepté ? Parce que le programmeur qui l'écrit prévoit une sortie — presque toujours un `break`. C'est un choix, pas un accident ; le motif est courant pour la boucle principale d'un jeu. Certains le trouvent moins lisible qu'une condition en tête : utilisé avec discernement, il peut au contraire clarifier.

C'est quoi, exactement, un nombre pseudo-aléatoire ? Un nombre produit par une formule. La suite n'est pas vraiment due au hasard, mais elle en a l'air — largement suffisant pour un jeu. (Des bibliothèques spécialisées font mieux quand il le faut.)

Faut-il toujours planter la graine ? Presque toujours pour un jeu. Mais parfois on veut *volontairement* la même suite à chaque exécution — pour tester ou déboguer, ou pour un dessin stable comme au cours 03.

Tous les jeux utilisent-ils la boucle de jeu ? La boucle de jeu est une *façon de lire* le déroulement d'un jeu, pas une obligation d'implémentation. Elle colle à presque tous — même quand le code ne contient pas une grande boucle unique.

Questions de réflexion

1. Qu'est-ce qui serait difficile à programmer sans boucles ?
2. Avantages et inconvénients du `switch` face à une suite de `if` / `else if` ?
3. Dans quel cas peut-on *volontairement* omettre le `break` d'un `case` ?
4. Quand préférer `while`, quand préférer `do` ?
5. Décris ton jeu préféré avec le vocabulaire de la boucle de jeu. Le schéma colle-t-il bien ?

Exercices

1. **Lire avant de lancer.** Qu'est-ce qui cloche dans cette boucle ? (Deux réponses possibles — pense à la règle « toute valeur a une vérité ».)

```
int x { 0 };
while (x)
{
    ++x;
    cout << x << endl;
}
```

2. **Le menu blindé.** Reprends le menu de difficulté (étape 4) et fais-le **redemander** le choix tant qu'il n'est pas valide : un `do … while` autour de la saisie et du `switch`, et une variable `bool choixValide` mise à jour dans les `case`. Le programme ne doit sortir qu'avec un choix entre 1 et 3.
3. **Le dé truqué.** Modifie le lancer de dé (étape 9) pour lancer **deux** dés, afficher chaque dé et leur somme, et commenter le résultat avec une suite de `else if` : « Snake eyes ! » pour un double 1, « Sept ! » pour une somme de 7, « Double ! » pour deux dés égaux, rien de spécial sinon. Attention à l'ordre des tests : un double 1 fait aussi un double.
4. **Les rôles inversés** *(plus costaud)*. Écris la version de « Devine mon nombre » où **c'est toi qui choisis** le nombre et **l'ordinateur qui devine**. À chaque proposition de l'ordinateur, tu réponds `+` (trop petit), `-` (trop grand) ou `=` (trouvé). Pour que l'ordinateur soit malin, fais-lui retenir les bornes `min` et `max` encore possibles et proposer le milieu — la stratégie de l'Essaie de l'étape 10.

Ce qu'il faut retenir

- Une expression s'évalue **vraie ou fausse** ; le type `bool` stocke ça, avec `true` et `false`. Toute valeur non nulle est interprétée comme vraie, zéro comme faux.
- Les comparaisons `==`, `!=`, `>`, `<`, `>=`, `<=` fabriquent du vrai/faux. `==` compare, `=` affecte — les confondre dans un `if` est le piège classique.
- `if` exécute si c'est vrai ; `else` couvre le sinon ; `else if` enchaîne des cas — le premier vrai gagne, le `else` final sert de filet.
- `switch` compare une valeur entière à des `case` ; `break` en sort, `default` attrape le reste. Sans `break`, l'exécution tombe dans le case suivant.
- `while` répète tant que c'est vrai (test **avant** le tour) ; `do` teste **après** — au moins un tour. Quelque chose doit pouvoir rendre le test faux, sinon boucle infinie.
- `break` sort d'une boucle, `continue` saute au tour suivant — avec modération.

- `&&` (ET) exige les deux ; `||` (OU) se contente d'une ; `!` (NON) inverse. Priorité `!` puis `&&` puis `||` ; des parenthèses claires valent mieux qu'une expression maligne.
- `rand()` tire un entier, `srand(static_cast<unsigned int>(time(0)))` plante la graine une fois au début, `% n + debut` ramène dans une plage — la version C++ standard de `ofRandom` / `ofSeedRandom` du cours 03.
- La **boucle de jeu** — mise en place, entrée, logique, affichage, fini ?, conclusion — décrit presque tous les jeux ; `setup` / `update` / `draw` d'openFrameworks en est une incarnation.

Ce cours est l'adaptation française du chapitre 2 de « Beginning C++ Through Game Programming » de Michael Dawson (Cengage Learning) : « Truth, Branching, and the Game Loop: Guess My Number ».